ARTI 502

Deep Learning: 2nd Assignment

Training-Testing CNN with CIFAR Dataset using Pytorch

Year 2026 Term 1

Group # 1

Group Members:

#	Member Name	Member Academic ID
1	Khalid Waleed Alhilal (Group Leader)	2230000788
2	Ali Khalid Albinali	2230003813
3	Mishary Waleed AlAwadh	2230006217
4	Nawaf Alsabr	2230004159
5	Yasir Saleh Alghamdi	2230004157
6	Ibrahim Alkuwaiz	2230002878
7	Meshal Alrubaie	2230005991

GitHub repository: https://github.com/ImSleepingHere/DL-Assignment2-CNN-CIFAR

Overview

In this assignment we train convolutional neural networks on the CIFAR-10 dataset (60,000 colour images of 32×32 pixels in 10 classes: 50,000 for training and 10,000 for testing) with PyTorch. We build a CNN out of the two core building blocks, **convolution layers** and **max-pooling layers**, write the training and test functions, look at the data, try every suggested data augmentation, record and plot train loss, test loss and test accuracy, and then keep improving the design (deeper networks and pretrained models) to push the test accuracy as close to 100 % as we could.

Objective

Implement two core building blocks of deep learning:

- Convolution layers
- Max-pooling layers

Summary of our results

All numbers below are copied from the executed notebooks in the repository (nothing is rounded up or estimated). Train loss and test loss include label smoothing 0.1, so they cannot go below about 0.5.

Model	Epochs	Final train loss	Final test loss	Test acc
Task 2 CNN: 6 conv + BN + MaxPool, GAP head (from scratch)	40	0.6620	0.7079	**92.94 %**
ResNet-18 + AutoAugment + CutMix (from scratch)	200	–	–	96.83 %
ViT-B/16 pretrained, full fine-tune	10	–	–	97.80 %
ViT-L/16 pretrained, frozen + linear classifier	30	–	–	98.05 %
RegNetY-32GF pretrained **CNN**, LP-FT (val-selected)	1 + 5	0.5213	0.5351	**98.62 %**
ViT-L/16 pretrained, LP-FT with layer-wise LR decay (best)	1 + 5	0.5039	0.5221	**99.31 %**

Best design: a ViT-L/16 pretrained on 3.6 billion images (SWAG), trained in two stages (linear probe, then fine-tuning of the whole network with layer-wise learning-rate decay): **99.31 %** test accuracy (69 errors out of 10,000). The best pure CNN is RegNetY-32GF with the same recipe: **98.62 %**. The best network trained from scratch is ResNet-18 (96.83 %), and the plain Task 2 CNN reaches 92.94 %.

Environment: Windows 11, VS Code + Jupyter, Python 3.14.4, PyTorch 2.14.0+cu130, torchvision 0.29.0. The Task 1–5 notebook and RegNet run were executed on an NVIDIA GeForce RTX 5080 (16 GB); the ViT runs were executed by our group member on an NVIDIA GeForce RTX 4080 SUPER (16 GB).

Source code: https://github.com/ImSleepingHere/DL-Assignment2-CNN-CIFAR

Tasks (with PyTorch)

Task 1: Getting the CIFAR Dataset

Instructions:

- Download the dataset first. Since PyTorch hosts the CIFAR dataset, you can load it using their helper function later.
- You need to change DATA_PATH to an empty directory to download the dataset to.

Deliverable:

- Screenshot showing the dataset loaded successfully.

Our Work & Results

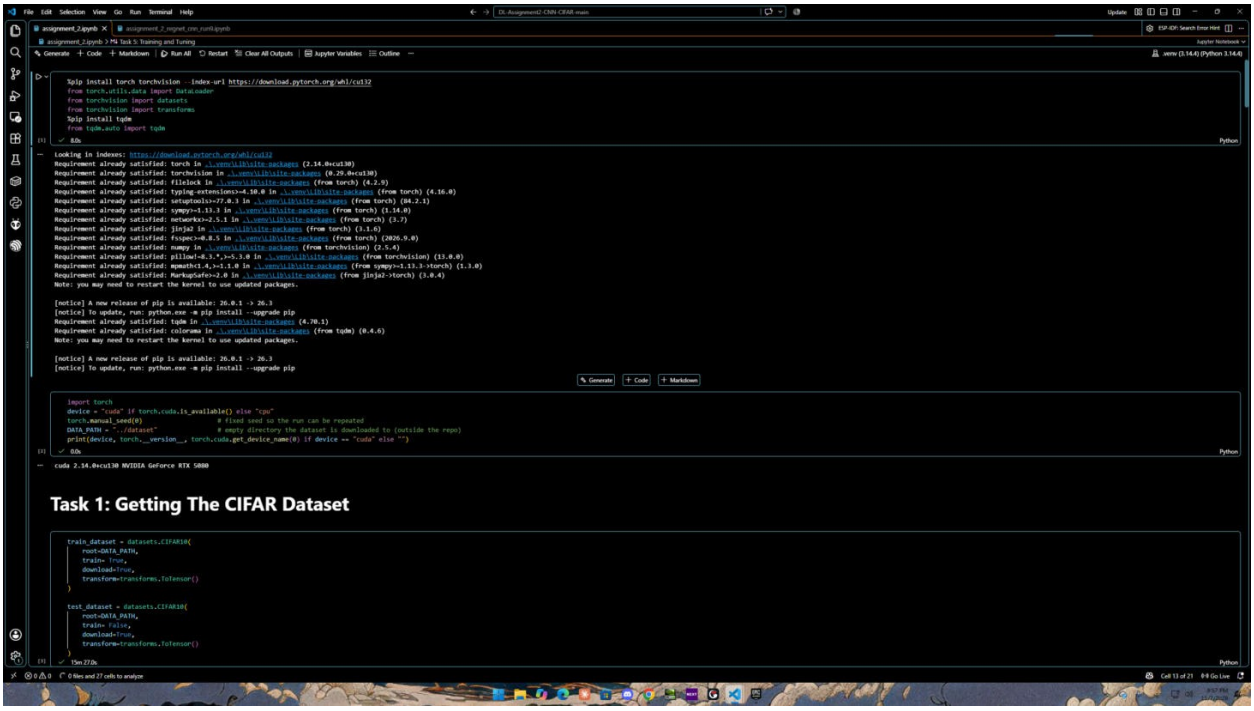


Figure 1: Imports, fixed seed, DATA_PATH and the GPU check (cuda, PyTorch 2.14.0+cu130, NVIDIA GeForce RTX 5080)

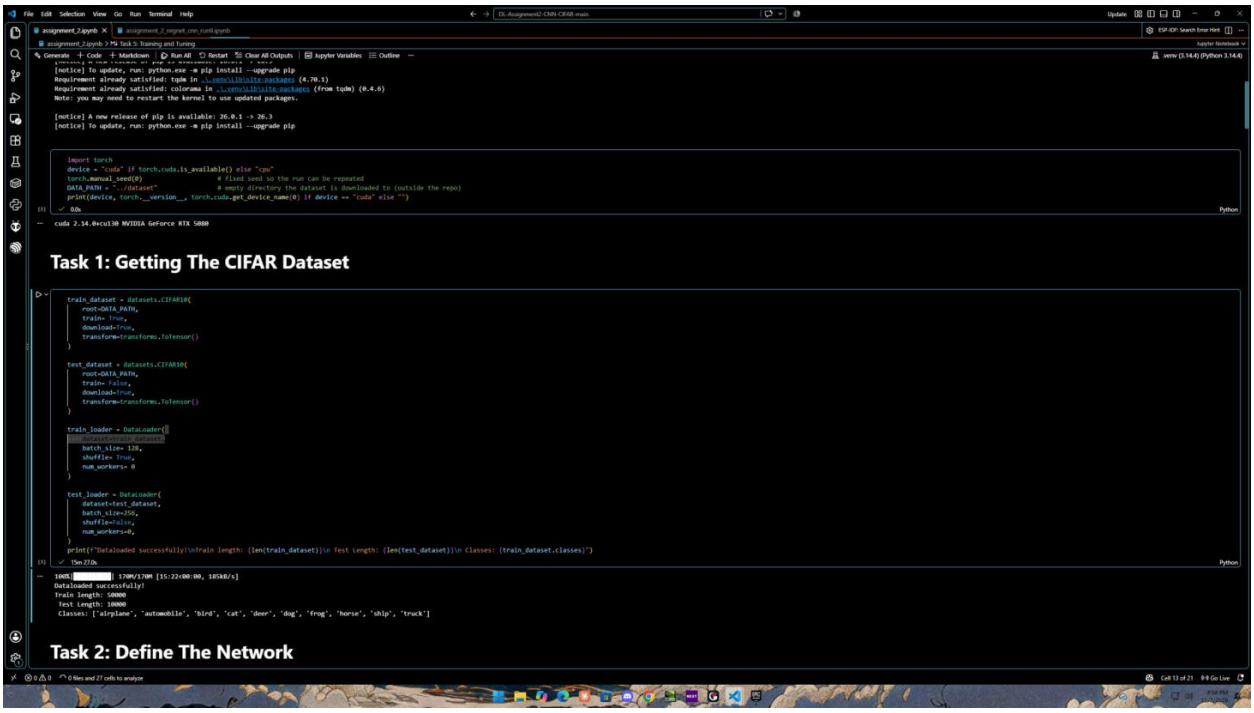


Figure 2: CIFAR-10 downloaded into the empty DATA_PATH directory (../dataset) and loaded successfully

Result: `DATA_PATH = "../dataset"` points to an empty directory outside the repository, so the 170 MB archive is not committed to GitHub. `datasets.CIFAR10(root=DATA_PATH, download=True)` downloaded and loaded the data: **50,000 training images** and **10,000 test images** in the 10 classes airplane, automobile, bird, cat, deer, dog, frog, horse, ship and truck.

Task 2: Defining the Network

Instructions:

- Just like with MNIST in the first assignment you need to define your network architecture. This time you will be using convolutional layers and maxpooling to extract features from our images before you feed those features into your final classifier.
- Check out the documentation for nn.Conv2d.
- So the first parameter is the number of channels in the input. Second is the number of filters we'll use (AKA number of channels in the output). Third is kernel size. Next is stride and padding which are optional and have default values.
- Create your training and test functions.

Deliverable:

- Provide screenshots of your code to define your network.
- Show screenshots of your training and test functions.

Our Work & Results

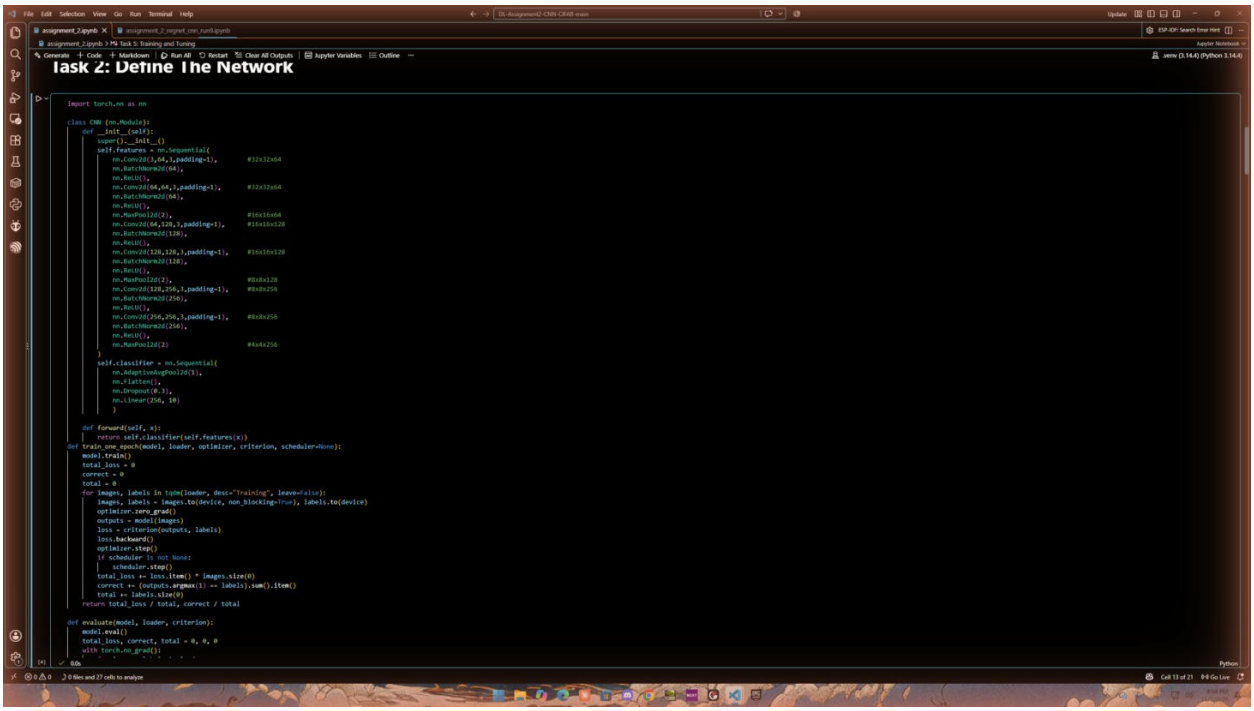


Figure 3: The CNN: three blocks of [Conv2d-BatchNorm-ReLU] × 2 + MaxPool2d, then global average pooling, dropout and a linear classifier

Architecture: every `nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1)` keeps the spatial size, and every `nn.MaxPool2d(2)` halves it: 3×32×32 → 64×32×32 → 64×16×16 → 128×16×16 → 128×8×8 → 256×8×8 → 256×4×4. BatchNorm after each convolution stabilises training. The classifier is `AdaptiveAvgPool2d(1)` → `Flatten` → `Dropout(0.3)` → `Linear(256, 10)`; averaging each feature map instead of flattening 4,096 values removes about 1 M parameters that would overfit.

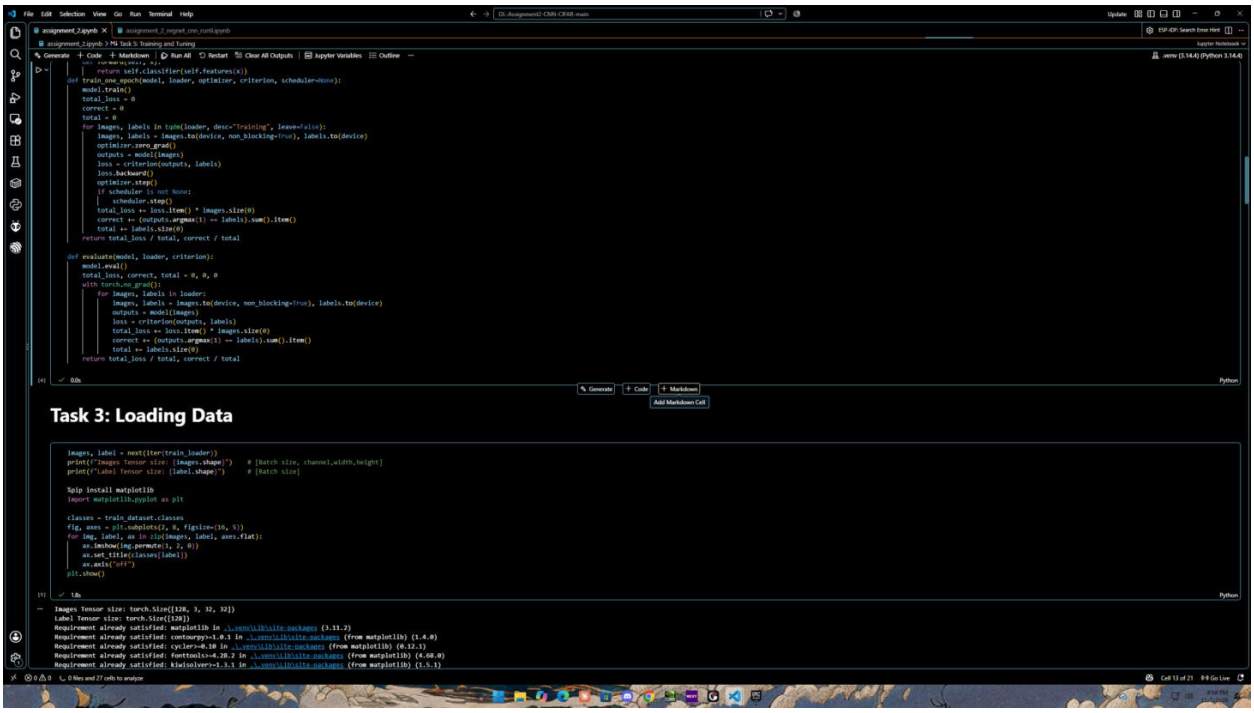


Figure 4: Training function train_one_epoch() and test function evaluate()

Training function `train_one_epoch()`: puts the model in training mode and, for every mini-batch, moves the images to the GPU, runs the forward pass, computes the loss, back-propagates (`loss.backward()`), updates the weights (`optimizer.step()`) and steps the learning-rate scheduler. It returns the average loss and the accuracy. **Test function** `evaluate()`: puts the model in eval mode (BatchNorm uses its running statistics, dropout is off) and computes loss and accuracy without gradients (`torch.no_grad()`).

Task 3: Loading Data

Instructions:

- It's worthwhile to check out how the dataloader loads the images into tensors. You can print out the size of the loaded data in the images tensors. Also you can use matplotlib to see the visualization of the images data.

Deliverable:

- Provide screenshots how to find:
  1. Images tensor size
  2. Data images visualization using matplotlib

Our Work & Results

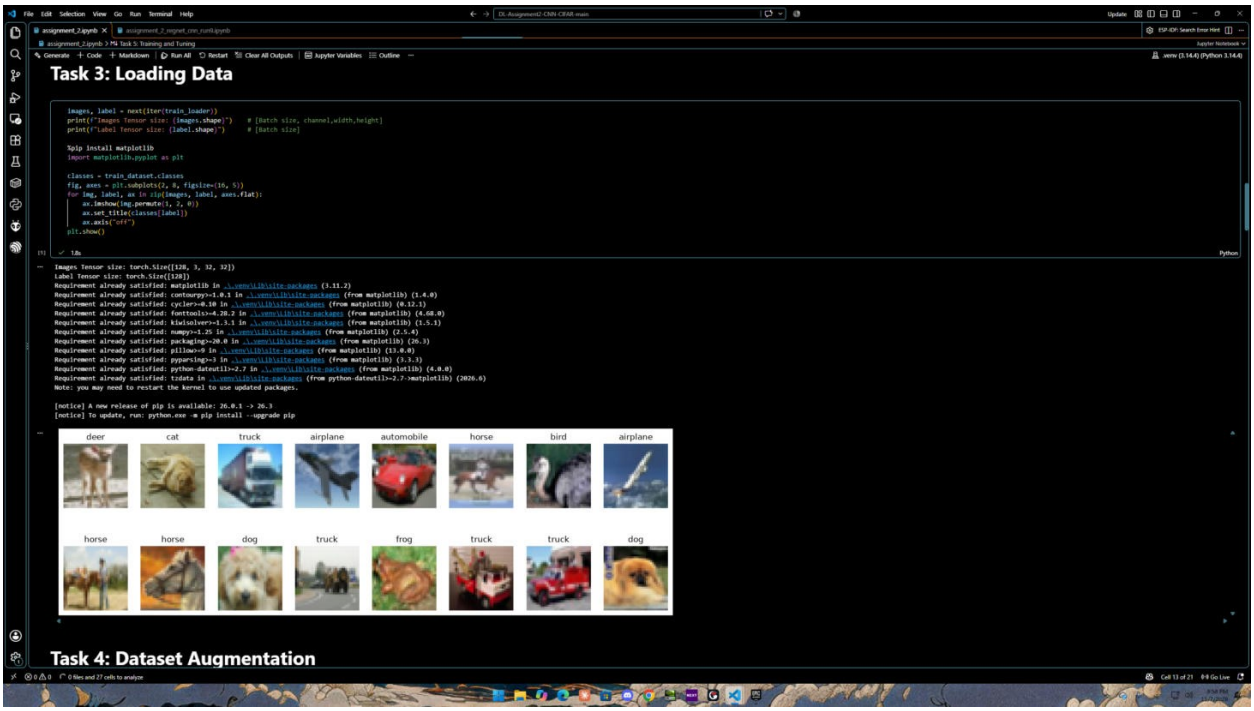


Figure 5: Tensor size of one mini-batch and 16 training images visualised with matplotlib

1. Images tensor size = [128, 3, 32, 32] = batch size × channels (RGB) × height × width; the label tensor is [128]. **2. Visualisation:** `img.permute(1, 2, 0)` changes C×H×W into H×W×C, the layout matplotlib's `imshow` expects; each image is titled with its class name.



Figure 6: One mini-batch of training images with their labels

Task 4: Dataset Augmentation

Instructions:

- In this assignment, you may find that data augmentation helps you a lot (or possibly hurts your performance).
- Play around with various data augmentations we will suggest some: ToPILImage, RandomHorizontalFlip, RandomResizedCrop, ColorJitter, RandomRotation, Normalize, adding various types of noise, ToTensor.
- Note: You should be careful about which of these you apply to the test data. You usually don't want to apply noise to the test data, but you do want to normalize it the same way for example.

Deliverables:

- Apply your code with various Dataset Augmentation as mentioned above.

Our Work & Results

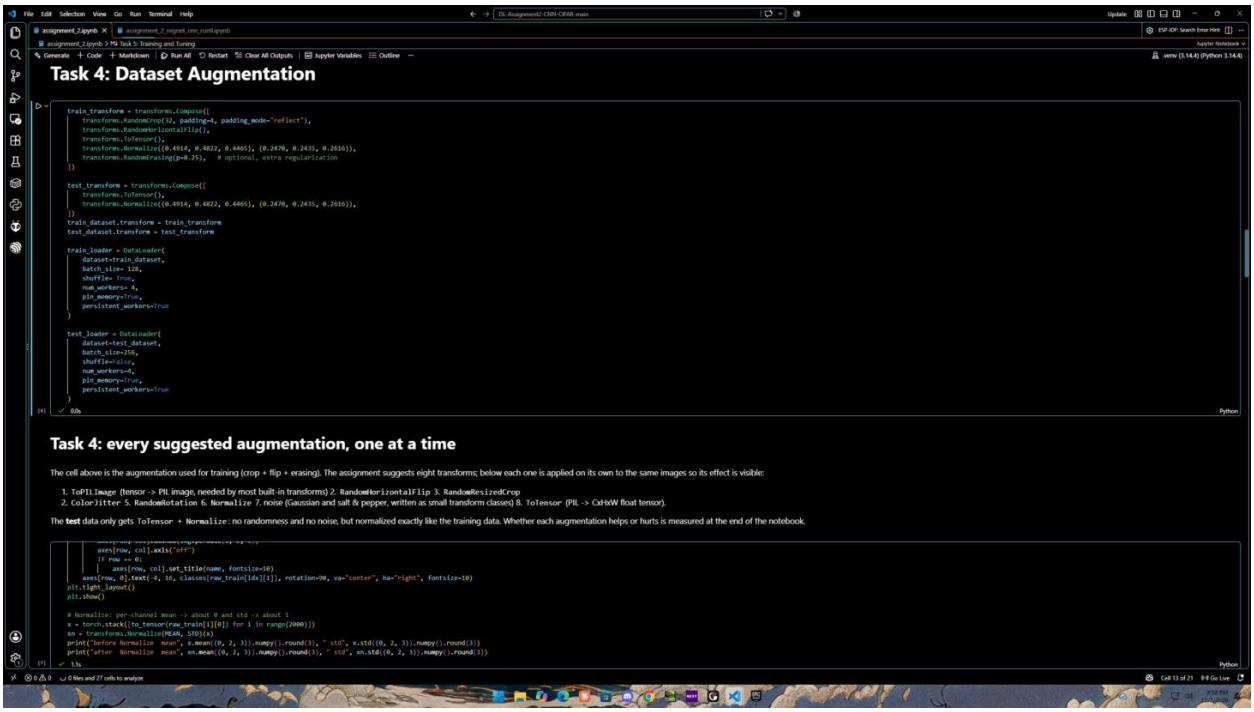


Figure 7: Training transform (random crop, flip, normalize, random erasing) and test transform (ToTensor + Normalize only)

Train vs. test: the training data gets random crop with 4-pixel reflect padding, horizontal flip, `ToTensor`, `Normalize` with the CIFAR-10 channel mean/std and `RandomErasing`. The test data only gets `ToTensor` and the **same** `Normalize`, with no randomness and no noise, exactly as the note in the instructions says.



Figure 8: All eight suggested augmentations, including two noise transforms we wrote (Gaussian and salt & pepper)

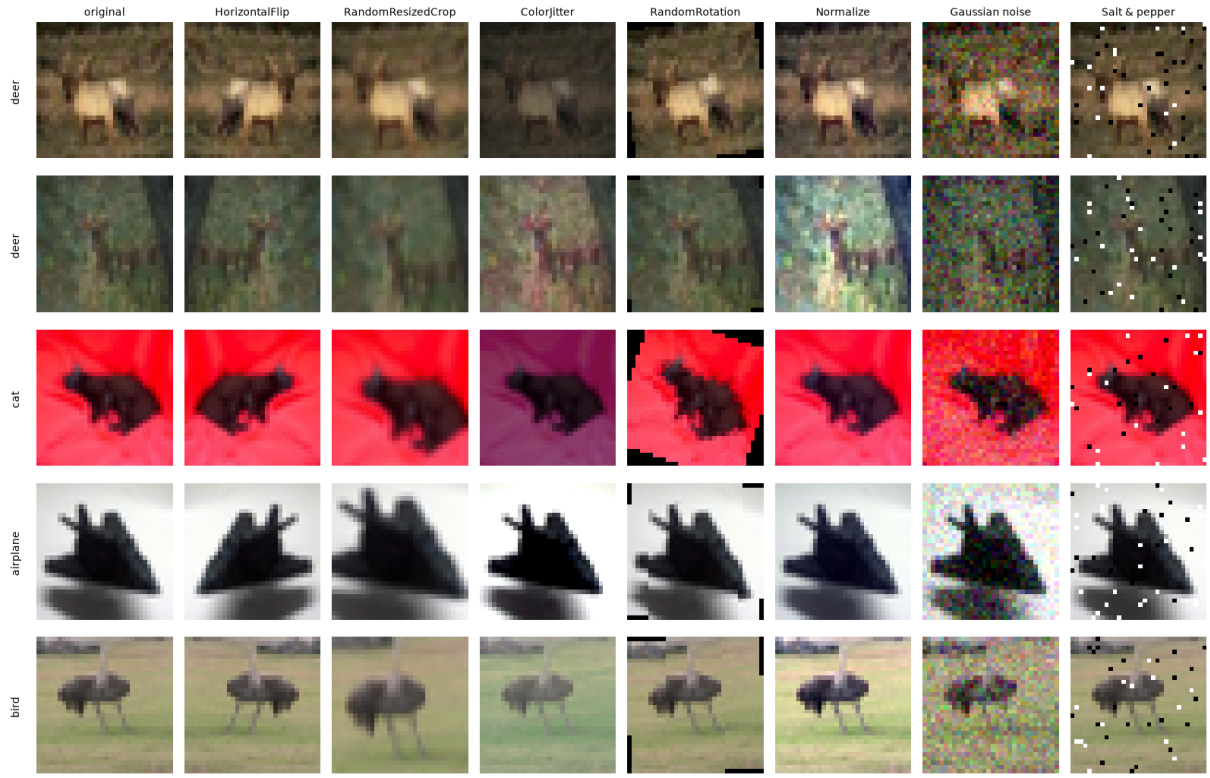


Figure 9: The same five training images under each augmentation

All eight suggestions are implemented: (1) `ToPILImage` converts the tensor to a PIL image, which the built-in transforms expect; (2) `RandomHorizontalFlip`; (3) `RandomResizedCrop`; (4) `ColorJitter` (brightness, contrast, saturation, hue); (5) `RandomRotation`; (6) `Normalize`; (7) noise: our own `AddGaussianNoise` (std 0.08) and `AddSaltPepperNoise` (4 % of pixels) classes; (8) `ToTensor` returns a C×H×W float tensor in [0, 1]. Normalize was checked numerically on 2,000 images: per-channel mean goes from [0.491, 0.484, 0.447] to [0.000, 0.006, 0.001] and std from [0.245, 0.242, 0.261] to [0.992, 0.995, 0.997].

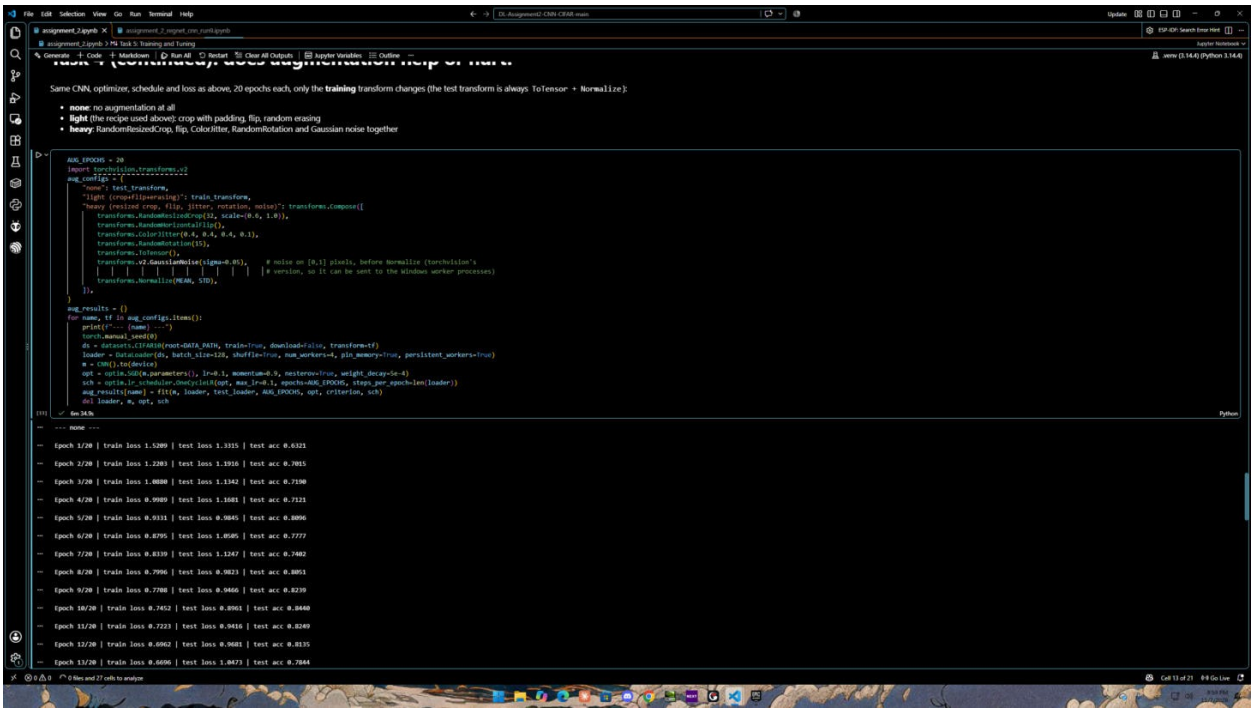


Figure 10: Does augmentation help or hurt? The same CNN is trained for 20 epochs with three different training transforms

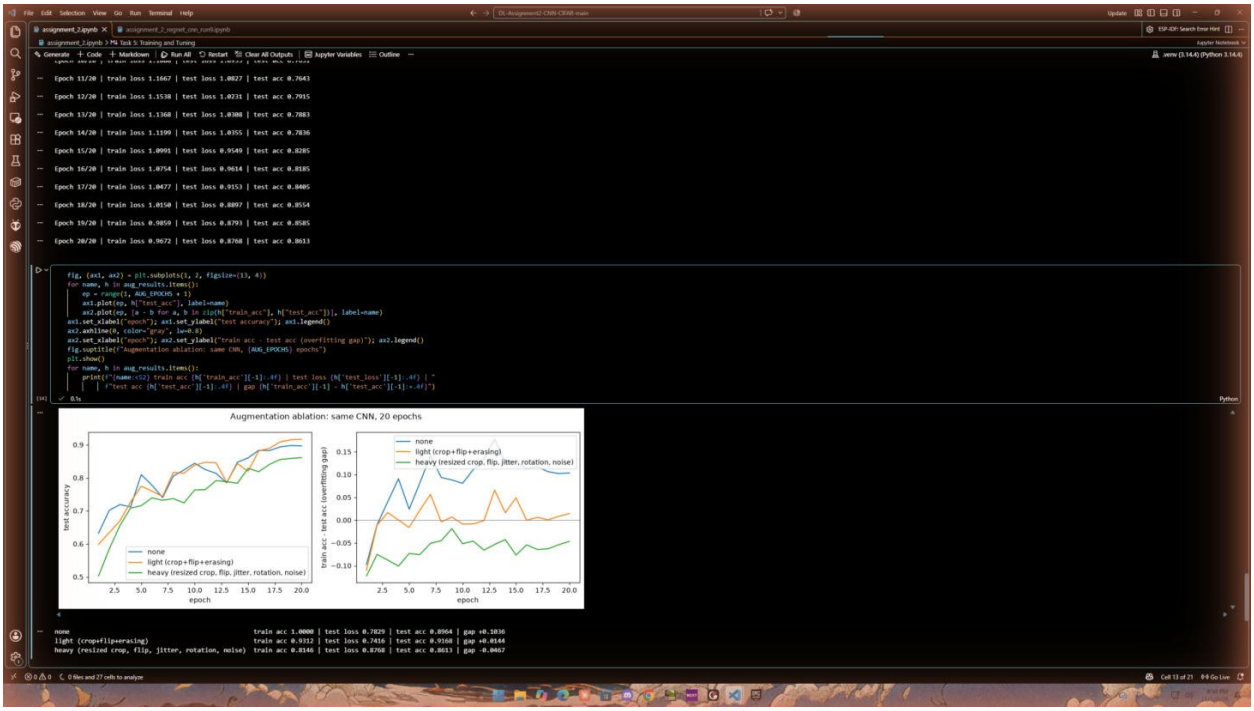


Figure 11: Test accuracy and overfitting gap (train acc − test acc) for each augmentation setting

Training augmentation (20 epochs, same CNN)	Train acc	Test loss	Test acc	Gap
None	100.00 %	0.7829	89.64 %	+10.36
Light: random crop + flip + random erasing	93.12 %	0.7416	**91.68 %**	+1.44
Heavy: resized crop + flip + jitter + rotation + Gaussian noise	81.46 %	0.8768	86.13 %	−4.67

Conclusion: augmentation can both help and hurt, as the assignment says. Without augmentation the network memorises the training set (100 % train accuracy, 10.4-point gap). Light augmentation closes the gap and gives the best test accuracy (+2.04 points). Heavy augmentation is too strong for this small CNN in 20 epochs: it under-fits (train accuracy below test accuracy) and test accuracy drops by 3.5 points. We therefore kept the light augmentation for the CNN.

Task 5: Training Function and Tuning hyperparameters

Instructions:

- Generally, it is useful to see how your training is going. Often people print their loss to make sure it goes down and their accuracy to make sure it goes up. But pictures are better than words. So for this part, you should record and plot the training loss, test loss, and test accuracy (and whatever else you want).
- Try different network architectures and experiment with hyperparameters.

Deliverables:

- Screenshot of your python source code that contains your training function with above requirements.
- What design that you tried worked the best? This includes things like network design, learning rate, batch size, number of epochs, and other optimization parameters, data augmentation etc.
- What was the final train loss? Test loss? Test Accuracy? Provide the plots for train loss, test loss, and test accuracy.

Our Work & Results

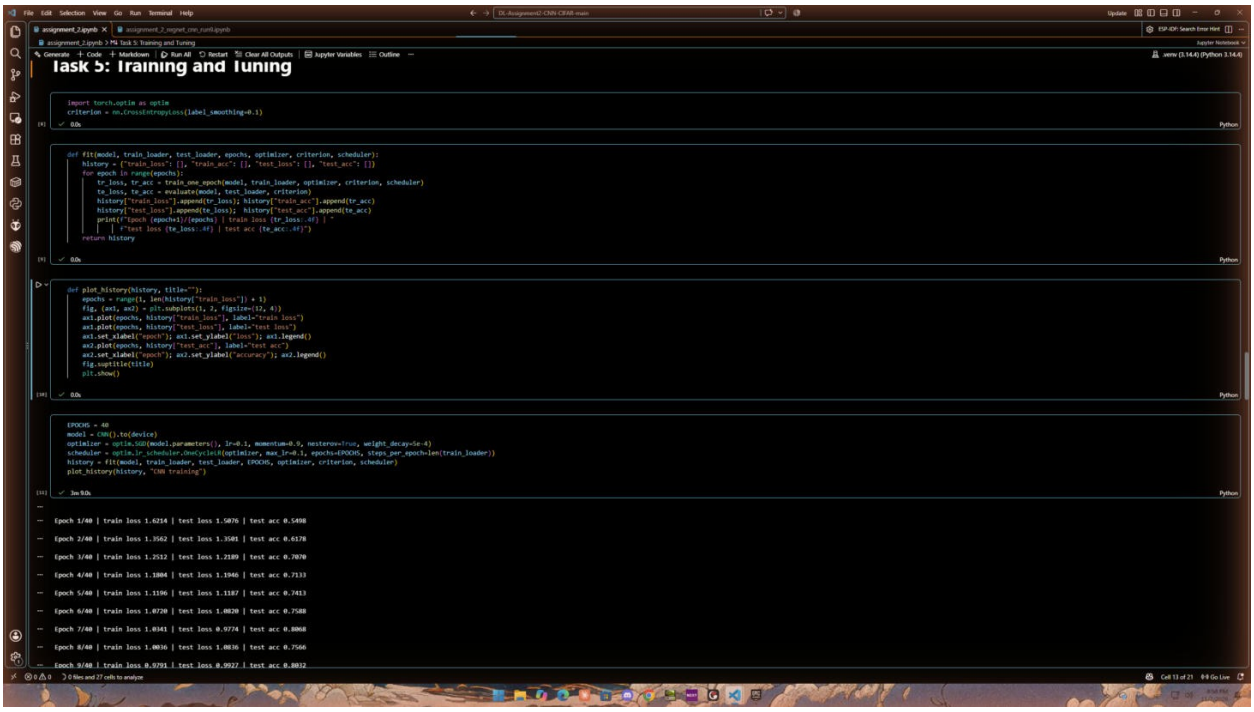


Figure 12: fit() records train loss, train accuracy, test loss and test accuracy every epoch; plot_history() plots them

`fit()` calls the training and test functions every epoch, stores the four values in a `history` dictionary and prints one debug line per epoch. `plot_history()` draws train loss vs. test loss and the test accuracy. Hyper-parameters for the CNN: SGD with Nesterov momentum 0.9, weight decay 5e-4, OneCycle learning-rate schedule with peak 0.1, batch size 128, 40 epochs, cross-entropy loss with label smoothing 0.1.

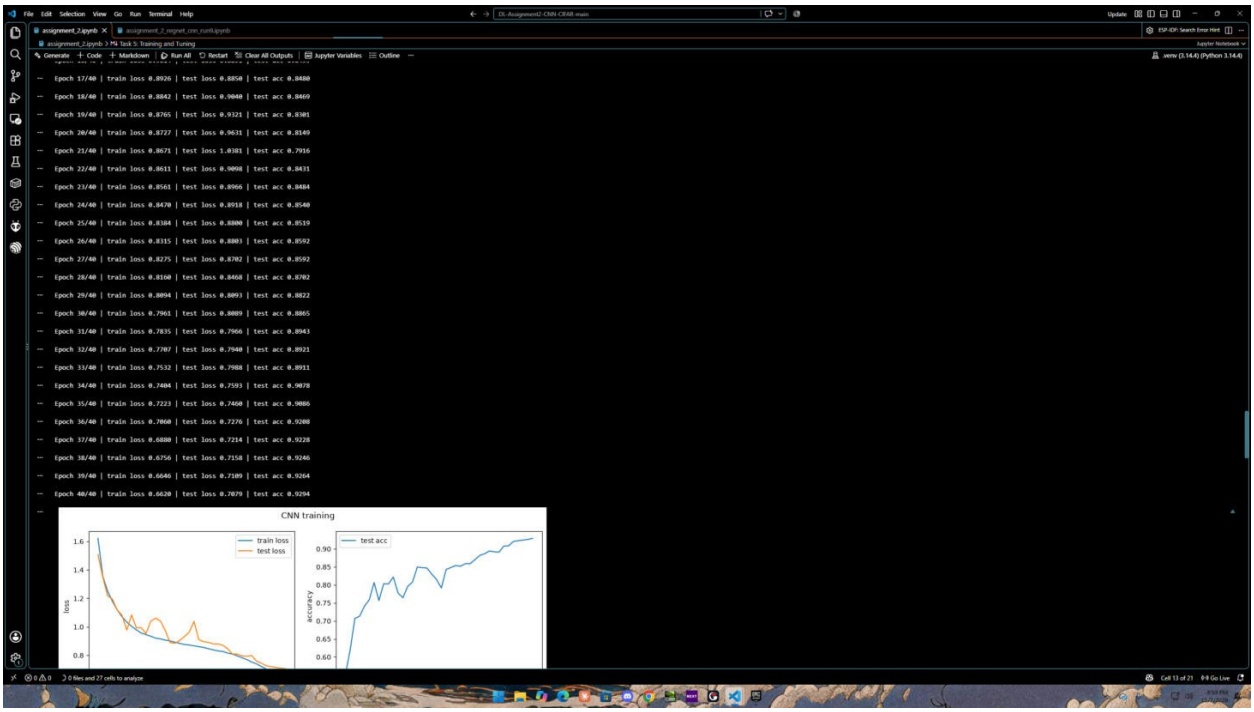


Figure 13: Training log of the CNN (40 epochs)



Figure 14: Train loss, test loss and test accuracy of the Task 2 CNN, and the final numbers

CNN result: final train loss **0.6620**, test loss **0.7079**, test accuracy **92.94 %** (the best epoch is the last one, epoch 40; 40 epochs took 3 min 9 s on the RTX 5080). Train and test loss stay close, so with light augmentation the CNN does not overfit.

Different architectures and hyper-parameters we tried

The full log with the reason for every change is in `EXPERIMENTS.md` in the repository. Each run kept what worked in the previous one:

Run	What changed	Test acc
1	Baseline CNN, Flatten + FC head, Adam lr 1e-3 constant, 30 epochs	88.21 %
2	SGD + Nesterov + weight decay, OneCycle LR, crop/flip/erasing, label smoothing, GAP head, 40 epochs (our Task 2–5 CNN)	92.94 %
3	ResNet-18 (skip connections), bf16 + channels_last, flip test-time augmentation (TTA), 60 epochs	95.74 %
4	ResNet-18 + AutoAugment + CutMix, 200 epochs (best from scratch)	96.83 %
5	ViT-B/16 pretrained (SWAG), full fine-tune, AdamW lr 5e-5, 10 epochs	97.80 %
6	ViT-L/16 pretrained, backbone frozen, only a linear classifier trained	98.05 %
7	ViT-L/16, linear probe 1 epoch, then fine-tune 3 epochs with layer-wise LR decay	99.28 %
8	**Same as Run 7 with 5 fine-tune epochs (best)**	**99.31 %**
9	RegNetY-32GF pretrained CNN, same LP-FT recipe, 45k/5k train/validation split	98.62 %

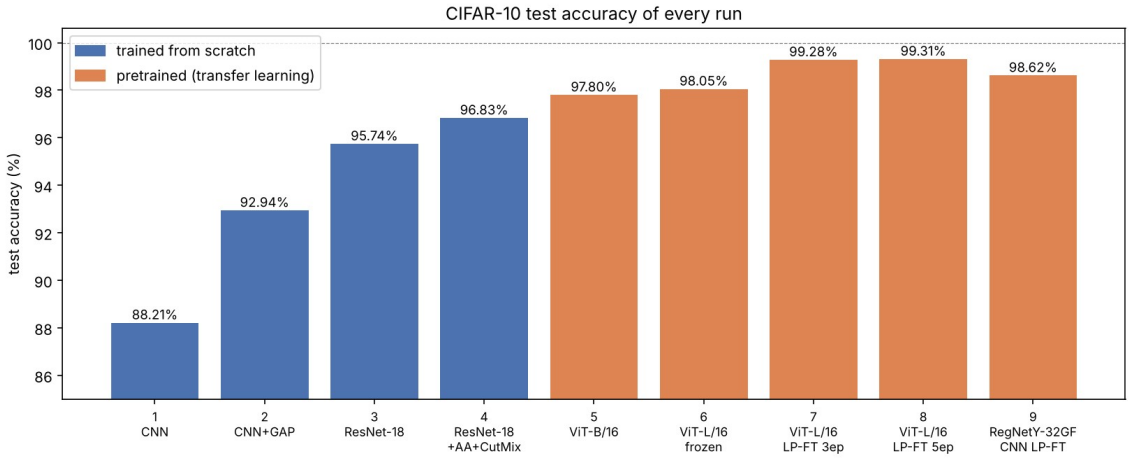


Figure 15: Test accuracy of every run

The design that worked best

- **Network:** ViT-L/16 (304 M parameters) with SWAG weights pretrained on 3.6 billion images, and a new zero-initialised 10-class linear head.
- **Data:** CIFAR images up-scaled 32 → 224 px (bicubic, on the GPU) and normalised with the ImageNet mean/std; augmentation = random crop + horizontal flip (light augmentation again worked best); flip TTA at test time.
- **Stage 1, linear probe:** backbone frozen, 1 epoch training only the head (AdamW lr 1e-3) → 97.79 %. A good head before touching the backbone avoids destroying the pretrained features (this is what went wrong in Run 5).
- **Stage 2, fine-tuning:** all layers, 5 epochs, AdamW, base LR 2e-5 for the head and top block, **layer-wise LR decay 0.8** per block (patch embedding gets 7.6e-8), weight decay 0.05, OneCycle with 10 % warm-up, gradient clipping 1.0, label smoothing 0.1, bf16 mixed precision.
- **Batch size:** 32, processed as 2 chunks of 16 with gradient accumulation to fit in 16 GB of GPU memory.

Final numbers of the best design: final train loss **0.5039**, test loss **0.5221**, test accuracy **99.31 %** with flip TTA (99.30 % without), i.e. 69 mistakes on the 10,000 test images. Per class: airplane 99.5, automobile 99.6, bird 99.4, cat 98.5, deer 99.3, dog 98.4, frog 99.8, horse 99.8, ship 99.8, truck 99.0 %. Cat and dog stay the hardest classes, as in every run.

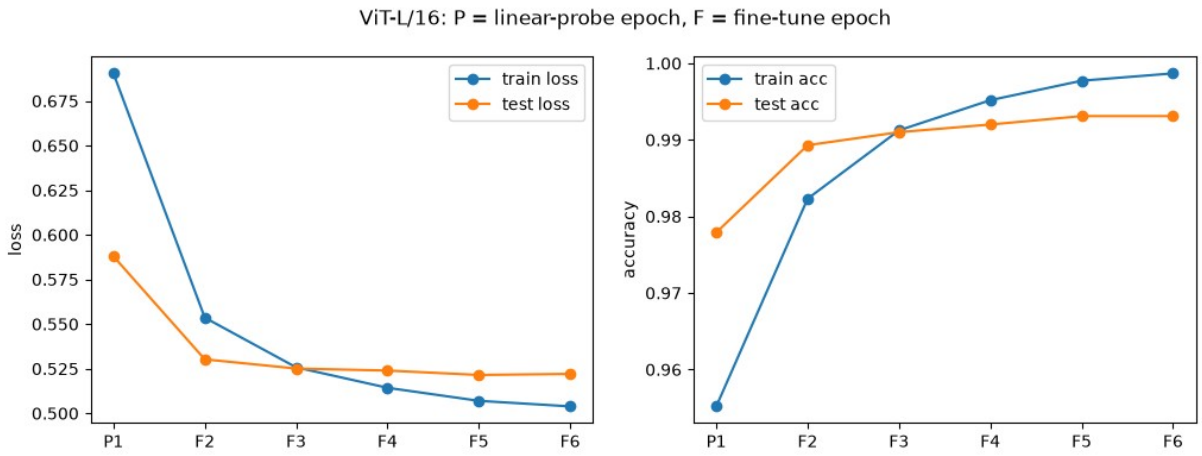


Figure 16: Best design (ViT-L/16 LP-FT): train loss, test loss and test accuracy for every epoch (P = linear-probe epoch, F = fine-tune epoch)

Best CNN: RegNetY-32GF (Run 9)

Because the assignment is about convolutional networks, we also applied the same two-stage recipe to the strongest pretrained CNN in torchvision, RegNetY-32GF (SWAG weights). This run holds out 5,000 training images as a **validation set**: the epoch is chosen on validation accuracy and the test set is only used to report the result, so this number is not tuned on the test set. Stage-wise LR decay 0.5 (stem → head), base LR 1e-4, 5 fine-tune epochs.

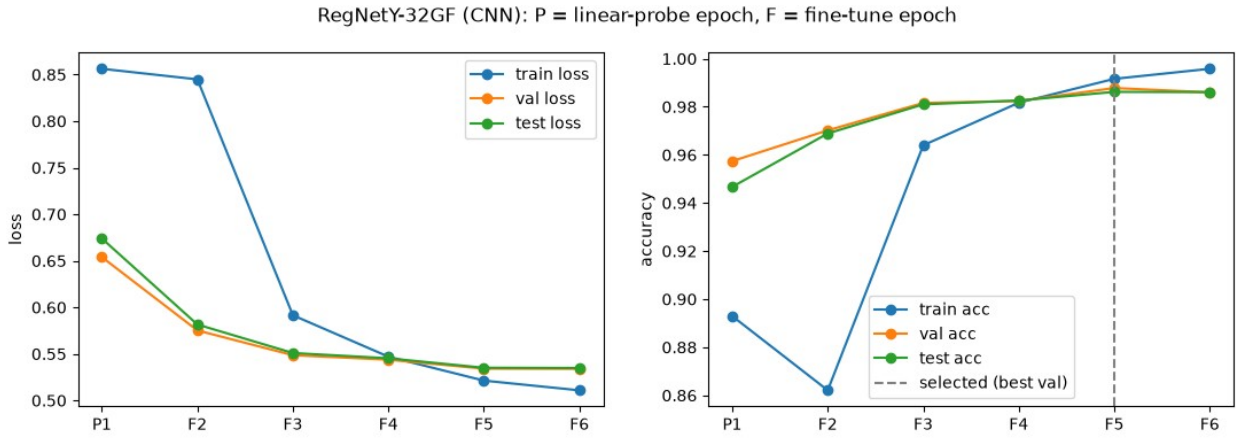


Figure 17: RegNetY-32GF: loss and accuracy per epoch on train, validation and test

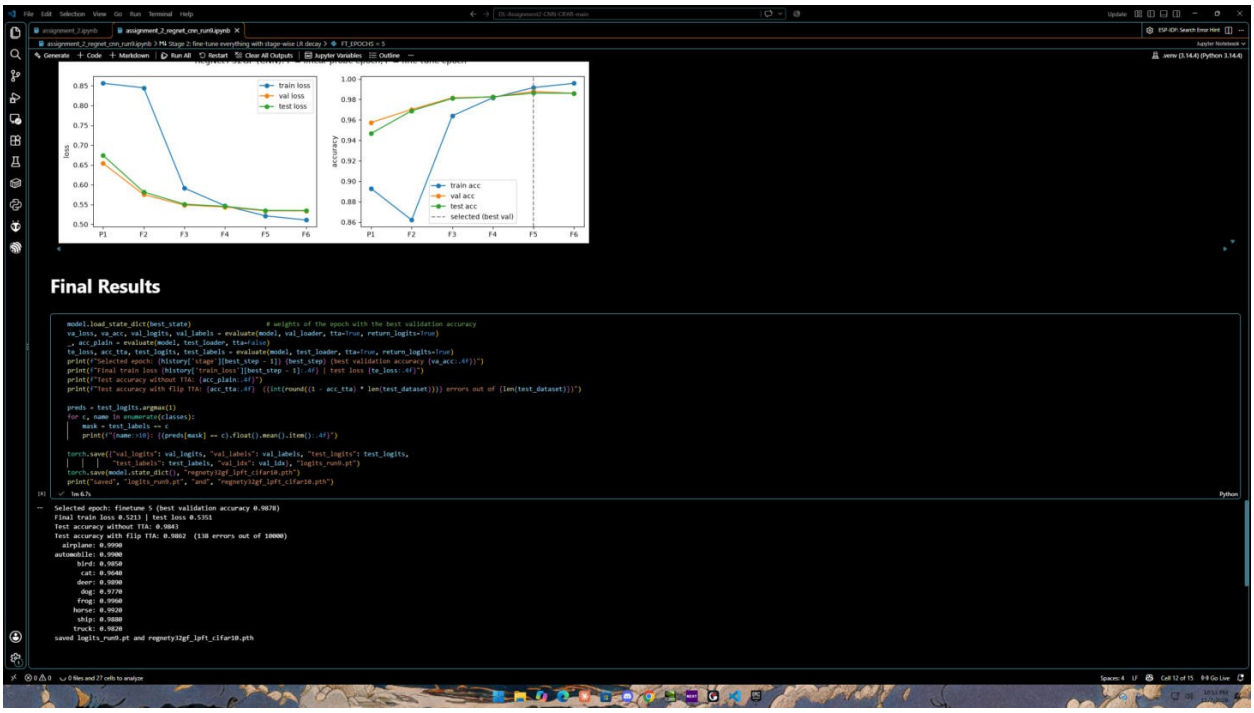


Figure 18: RegNetY-32GF final results on the test set

Run 9 result: selected epoch 5 of 6 (the 4th fine-tune epoch) (validation accuracy 98.78 %); final train loss **0.5213**, test loss **0.5351**, test accuracy **98.62 %** with flip TTA (98.43 % without), 138 errors out of 10,000.

Error analysis: which images are still wrong?

Computed from the saved test-set predictions of Run 9 (`logits_run9.pt`), so it uses exactly the model reported above.

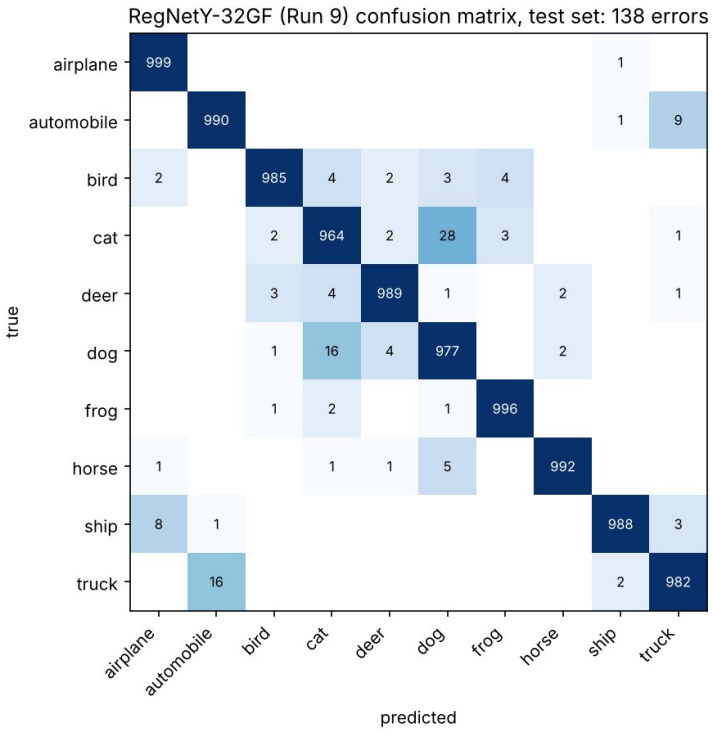


Figure 19: Confusion matrix of the best CNN on the 10,000 test images (colour on a log scale)

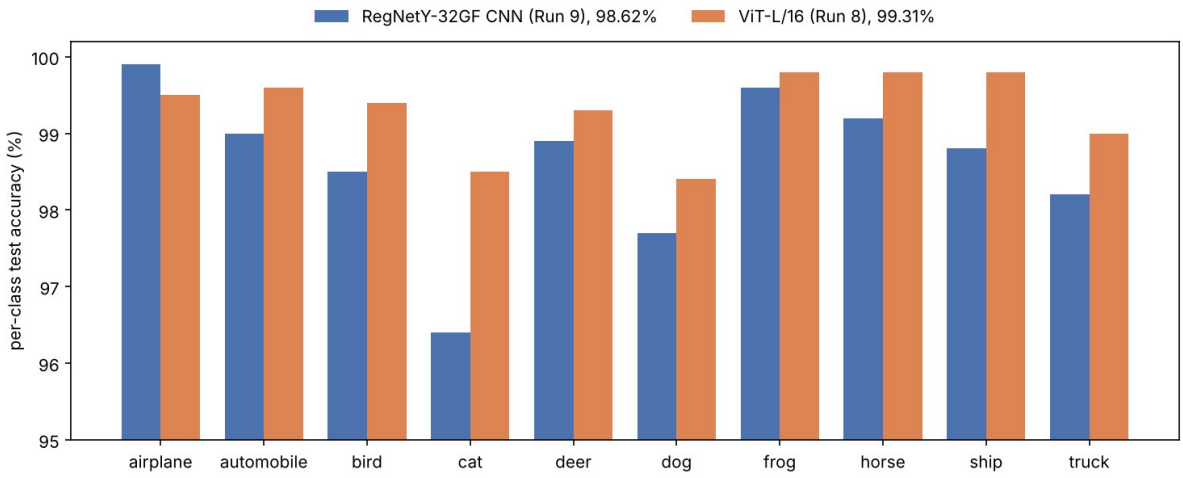


Figure 20: Per-class test accuracy of the two best models

Most frequent mistake (true → predicted)	Count	Share of the 138 errors
cat → dog	28	20.3 %
truck → automobile	16	11.6 %
dog → cat	16	11.6 %
automobile → truck	9	6.5 %
ship → airplane	8	5.8 %

Findings: cat ↔ dog alone is 44 of the 138 errors (32 %) and truck ↔ automobile another 25 (18 %): the remaining mistakes are between classes that look alike at 32×32 pixels, not random failures. For both models cat and dog are the two hardest classes. The model also knows when it is unsure: the mean softmax confidence is 0.906 on correct predictions but 0.715 on wrong ones, and the 87.3 % of test images predicted with confidence ≥ 0.9 are 99.71 % correct. (Label smoothing 0.1 caps the confidence at about 0.91, which is why no prediction reaches 0.95.)

Why not exactly 100 %? The best published CIFAR-10 results are around 99.5 %. A few test images are ambiguous or wrongly labelled, so 100 % is not a realistic target; our 99.31 % is close to that limit.

Task 6: Save your source code

Instructions:

- Save your python source code.

Deliverable:

- Submit your Github link of your source code along with this document, which includes all screenshots via Blackboard.

Our Work & Results

GitHub repository: https://github.com/ImSleepingHere/DL-Assignment2-CNN-CIFAR

The whole project is Jupyter notebooks:

- `assignment_2.ipynb`: Tasks 1–5 (dataset, CNN, train/test functions, data visualisation, all augmentations and the augmentation experiment, training with plots).
- `assignment_2_resnet_run3.ipynb`, `assignment_2_resnet_run4.ipynb`: ResNet-18 from scratch.

- `assignment_2_vit_run5.ipynb`, `assignment_2_vitl_linear_run6.ipynb`, `assignment_2_vitl_finetune_run7.ipynb`, `assignment_2_vitl_finetune_run8.ipynb`: pretrained vision transformers (Run 8 is the best design).
- `assignment_2_regnet_cnn_run9.ipynb`: RegNetY-32GF pretrained CNN with a validation split.
- `EXPERIMENTS.md` (what was changed between runs and why), `README.md`, `requirements.txt`, and this report with all screenshots in `report/`.

The dataset (`../dataset`) and the trained weights (`*.pth`, hundreds of MB to GBs) are excluded with `.gitignore`; the notebooks download the data and recreate the weights.

Hardware & Environment

Component	Specification
GPU (Tasks 1–5, augmentation experiment, Run 9)	NVIDIA GeForce RTX 5080, 16 GB (15.9 GiB detected by the notebook, Figure 1)
GPU (Runs 3–8)	NVIDIA GeForce RTX 4080 SUPER, 16 GB (detected in the Run 8 notebook)
Software	Windows, VS Code + Jupyter, Python 3.14.4 (.venv), PyTorch 2.14.0+cu130, torchvision 0.29.0+cu130
Reproducibility	torch.manual_seed(0); Run 9 train/validation split fixed with seed 0; exact package versions in requirements.txt

Step (measured in the notebooks)	Time	Peak GPU memory
CIFAR-10 download (170 MB at 185 kB/s)	15 min 22 s	–
Task 2 CNN, 40 epochs (Task 5 training)	3 min 9 s	–
Augmentation experiment, 3 × 20 epochs (Task 4)	6 min 35 s	–
Run 9 RegNetY-32GF: 1 linear-probe epoch	5.1 min	6.0 GiB
Run 9 RegNetY-32GF: 5 fine-tune epochs	29.7 min (5.7–6.5 min each)	6.0 GiB
Run 8 ViT-L/16 fine-tune (from EXPERIMENTS.md)	about 6.7 min per epoch	about 10.5 GB

The small CNN trains in about 5 seconds per epoch, so all of Tasks 1–5 run in under 30 minutes including the download. The pretrained models are about 130–270× larger (1.1 M parameters vs. 145 M and 304 M) and need about 6 minutes per epoch at 224×224 pixels, which is why they were trained for only 1 + 5 epochs.

Grading Rubric

Criterion	Excellent	Good	Developing	Poor	Marks
Task 1: Getting the CIFAR Dataset	Dataset loaded correctly with clear screenshot showing successful execution	Loaded correctly but screenshot is unclear	Dataset loaded with issues or incomplete evidence	Not loaded / missing screenshot	0.5
Task 2: Defining the Network	All clear screenshots	Minor mistakes or missing small details	Several missing description or unclear screenshots	Not answered / incorrect / missing evidence	2
Task 3: Loading Data	All clear screenshots	Minor mistakes or missing small details	Several missing description or unclear screenshots	Not loaded / missing screenshot	0.5
Task 4: Dataset Augmentation	All datasets augmentation implemented	Major datasets augmentation implemented	Minor datasets augmentation implemented	Not done	2
Task 5: Training Function and Tuning parameters	Tuning parameters and model architecture created, screenshot included, and results description explained accurately	Tuning parameters and model architecture correct but results description is weak	Tuning parameters and model architecture correct but incorrect results description	Not done	2
Task 6: Save your source code	Source code saved successfully	Source code saved successfully but minor missing item	Source code save incomplete or file format incorrect	Not submitted / missing	0.5
Total					7.5